World Wide Web (2018) 21:803–823
DOI 10.1007/s11280-017-0479-4

Channel dynamic adjustment in data broadcast

Wenbin Hu [1] · Zhenyu Qiu [1] · Cong Nie [1] · Bo Du [1]

Received: 23 January 2017 / Revised: 18 May 2017 / Accepted: 11 July 2017 /
Published online: 8 August 2017
© Springer Science+Business Media, LLC 2017

Abstract Data broadcasting has become the preferred method to dispense data to a large number of mobile users. Current researches on on-demand data broadcast mainly propose algorithms based on a single broadcast channel or fixed multi-channel, i.e., fixed channel model. As a result of the dynamic diversity of data characteristics and client demands, the fixed channel model faces significant challenges in parallel broadcast diverse data. Further, the dynamic adjustment of the broadcast channel (dynamic channel model) based on client requests is favorable to service quality because it determines the number and sizes of channels that adapt to client demand in real-time. However, the dynamic channel model has not yet been thoroughly investigated for on-demand wireless data broadcasts. Accordingly, in this paper, a channel dynamic adjustment method (CDAM) is proposed. The innovations behind CDAM lie in three aspects. First, a data item priority evaluation and selection algorithm (S-RxW/SL) is proposed for evaluating the priority of data items and selecting the high priority data items to be considered in a broadcast cycle. Second, a weight and size average cluster algorithm (WSAC) is proposed for mining data item characteristics and clustering them. Third, based on the clustering results of WSAC, a channel splitting and data allocation algorithm (CSDA) is proposed for dynamically splitting the channel and allocating data items to the corresponding sub-channel. We compare the proposed method with some state-of-the-art scheduling methods through simulation. The theoretical findings and simulation results reveal that significantly better request loss rate (LR) can be obtained by using our method as compared to its alternatives.

Keywords Dynamic channel model · Characteristic mining · Data broadcasting · Data priority

This article belongs to the Topical Collection: *Special Issue on Mobile Crowdsourcing*
Guest Editors: Bin Guo, Xing Xie, Raghu K. Ganti, Daqing Zhang, and Zhu Wang

✉ Wenbin Hu
hwb@whu.edu.cn

[1] School of Computer, Wuhan University, Wuhan, Hubei Province, China

1 Introduction

With the rapid growth of mobile data services, wireless data broadcast has become a popular solution for propagating information in mobile communication because it has the advantage of channel sharing technology to allow an arbitrary number of clients to access data simultaneously [3]. It is suitable for data services in asymmetric communication environments, such as news reports, stock quotes, flight schedules, traffic reports etc. [13, 19, 22]. Depending on whether the client has the ability to submit requests, broadcast methods can be divided into two categories—push-based broadcast and on-demand broadcast. Push-based data broadcast considers a static data access pattern and disseminates data items cyclically according to a pre-defined schedule, which is inappropriate in an environment where the access pattern changes dynamically. In contrast, the server only broadcasts the requested data items in on-demand data broadcast. On-demand data broadcast combines the merits of broadcasting and on-demand services, and is the current research hotspot [14, 20, 21, 24, 31, 32].

On-demand data broadcast has been proposed to facilitate dynamic broadcast in large-scale wireless networks. The existing methods for on-demand data broadcast mainly focus on the fixed channel model, wherein the number and bandwidths of channels are immutable. Specifically, the fixed channel model has two different architectures: the single channel architecture [11] and multi-channel architecture [8]. In single channel architecture, the server broadcasts data using a fixed downlink channel. Some excellent scheduling algorithms have been proposed in single channel architecture, including classical algorithms— earliest deadline first (EDF) [35], RxW [1] etc.—and state-of-the-art algorithms—LxRxW [12]. In fixed multi-channel architecture, the data is transmitted by a number of fixed bandwidth downlink channels. Fixed multi-channel architecture has the advantages of parallel broadcasting and suitability for multi-item data broadcasting. Excellent scheduling algorithms, such as scheduling scheme with maximum gain heuristic (SLLH) [23], maximum paid cost first (MPCF) [17], have been proposed for fixed multi-channel architecture. These algorithms have achieved promising results in reducing the average access time (AAT) and request loss rate (LR). However, along with the increase in the diversity of data characteristics and client demands, the problems illustrated in Figure 1 cannot be ignored. On the one hand, the single channel

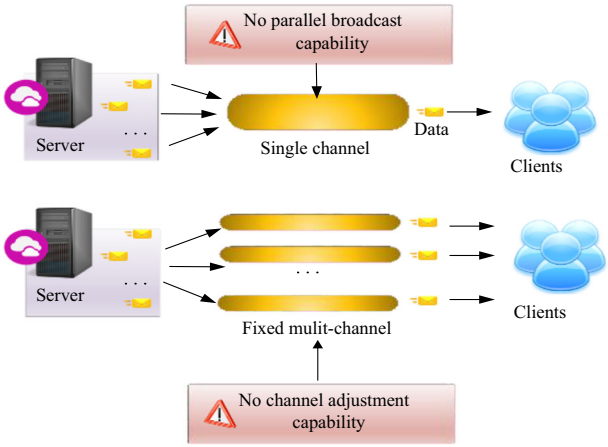

Figure 1 The problems of fixed channel model caused by dynamic diversity of data characteristics and client demands

architecture has no capability of parallel broadcasting and is not practical for real-world applications. On the other hand, the fixed channel architecture is designed for a particular wireless network and has no capability of channel adjustment; consequently, it is not suitable for an environment with changing data characteristics and client demands.

Based on the above problems, we claim that it is necessary and possible to investigate the dynamic channel model to address these challenges. One the one hand, the limitation of the physical resources (e.g., mobile devices) usually leads to a single channel of high-speed transmission not workable. One the other hand, dynamic channel model allows the broadcast system to adapt to fluctuating mobile networks. In this study, we focused on the problem of optimal dynamic channel adjustment (ODCA) and proposed a channel dynamic adjustment method (CDAM). The main aspect of CDAM is mining the characteristics of the data items, clustering the data items by their characteristics, and finally, splitting the channel and allocating data items using the clustering results. The main contributions of this paper are as follows.

(1) To improve the accuracy of characteristic mining, a data item priority evaluation and selection algorithm (S-RxW/SL) is proposed for evaluating the priority of data items and selecting the high priority data items to be considered in a broadcast cycle.
(2) A weight and size average cluster algorithm (WSAC) is proposed for mining data item characteristics and clustering them. WSAC synthetically considers the data item size, channel size, and data item priority.
(3) Based on the results of WSAC, a channel splitting and data allocation algorithm (CSDA) is proposed for dynamically splitting the channel and allocating data items to the corresponding sub-channel.

The rest of this paper is organized as follows: Section 2 presents the related works. Section 3 presents the preliminaries of the problem. Section 4 presents the CDAM. Finally, Section 5 provides the simulation results and Section 6 concludes this paper.

2 Related works

The fixed channel model has two different architectures: the single channel architecture and multi-channel architecture. Many scholars have performed research on these two architectures, and a number of classical algorithms have been proposed.

The typical researches on single-channel architecture are as follows. Xuan [35] proposed a broadcast on-demand model (BoD) based on the EDF algorithm, which can ensure bandwidth utilization and effectively control the access time of clients. Fang [7] proposed a deadline-constrained model and an aggregated efficient request (ACR) algorithm to satisfy the client requests before the deadline of the data item. Kalyanasundaram [15] considered the request length and request deadline, and proposed scheduling BCast and BCast2 for certain and uncertain request lengths. Ng [27] proposed the most requests served (MRS) broadcasting model and variants with caching strategies for a real-time information transmission system. MRS considers the sensitivity of the data item size to the bandwidth on the basis of most requests first (MRF) [6]. Dewri [5] considered the deadline constraint and asserted that the response time of the requests, along with the priority of the requests, should determine the efficiency of scheduling. Aksoy and Franklin [1] proposed the RxW algorithm considering the number of requests and the longest waiting time. Wu

and Lee [34] proposed the slack time inverse number of pending requests (SIN-α) algorithm with a deadline constraint, which optimizes the system efficiency by a request deadline. Hu [12] proposed the LxRxW algorithm, which considers the number of lost requests during the broadcast of the subsequent item, number of requests, and waiting time. All the above algorithms assume that the size of the data items is fixed.

The typical researches on fixed multi-channel architecture are as follows. FLAT [28], which randomly allocates data items to channels, is the simplest push-based multi-channel broadcasting algorithm. Anticaglia [2] studied the methods to minimize the average response time of multiple broadcast channels and proposed a new heuristic strategy called GREEDY, which combines a novel characterization with the known greedy approach and optimally partitions data among the channels. Waluyo [31] focused on the broadcast queue in the server to minimize the access time. Chung [4] studied multi-channel on-demand data broadcast with a deadline constraint and deduced the minimum number of channels required in such a multi-channel broadcast system. Lee et al. [16] provided an analytical model and cost formulation for exclusive broadcast channels and exclusive on-demand channels. They also derived a cost model for dynamic channel allocation methods and proposed a channel adaptation algorithm for optimizing the system performance. Ali et al. [26] introduced an admission control scheme called item level admission control (ILAC) for a multi-channel data broadcast system, which ensures that clients can be informed in a timely manner. ILAC regards multi-item queries with a deadline as multiple dummy single-item queries with the same deadline, each of which consists of one of the requested data items in the multi-item query. Zheng et al. [36] concentrated on data allocation methods for multi-channel broadcasting. They considered the data item access frequencies, data item lengths, and bandwidth of the different channels to develop the near-optimal scheduling algorithm TOSA. TOSA is based on the idea of two-level data allocation, i.e., a high-level optimization step for allocating the data to the channels, and a low-level optimization step to schedule the data within a channel. The above scheduling algorithms are all for push-based broadcast, however, the on-demand data broadcast for fixed multi-channel architecture has not yet been thoroughly investigated. Lu [23] studied the maximum throughput request selection problem (MTRS) and the minimum latency request order problem (MLRO), and proposed a two-stage scheduling scheme called the SLLH to arrange the requested data items in order to minimize the average access latency. Lei [17] considered both LR and AAT and defined $Cost_{d_i}$ in terms of the cost of the data item d_i not being broadcast during the current time slot, and subsequently proposed the MPCF algorithm, which chooses the data item with the largest cost to broadcast first, where $Cost_{d_i}$ is determined by LR and AAT.

In conclusion, various scheduling algorithms have been proposed to improve the performance of on-demand data broadcast system for the fixed channel model, but there is no scheduling algorithm that focuses on the dynamic channel model to address the challenges resulting from the dynamic diversity of data characteristics and client demands.

3 Preliminaries

In this section, we first describe the system architecture of an on-demand data broadcast with the dynamic channel model and the related notations. Thereafter, we introduce the performance metrics. Finally, we present the problem statements.

3.1 System architecture and notation

The system architecture of an on-demand data broadcast system with the dynamic channel model is depicted in Figure 2. We assume that the system consists of a server and a number of clients that use an uplink channel to request a data item from the server. The server broadcasts data items within a set of physical broadcast channels, where the bandwidths and number of channels are dynamic. The server maintains a *request queue*, *pending queue*, and set of *broadcast queues*. The clients send the requests to the server via an uplink channel with a deadline and subsequently listen to the downlink channels until the desired data item is broadcast or the deadline is exceeded. We assume that the clients can listen to all the channels simultaneously.

As depicted in Figure 2, the system model consists of the following four steps—(1) Accept: The server receives the client request and saves it in the *request queue*; (2) Fetch: The server fetches the data from the database to generate the *pending queue* according to the *request queue*; (3) Priority evaluation and clustering: The server evaluates the priority of data items using the S-RxW/SL, and subsequently clusters the data items using the WSAC; (4) Channel splitting and data allocating: The server dynamically splits the channel and allocates data items to the corresponding sub-channel using the CSDA algorithm.

We present the following definitions of notations for an on-demand data broadcast system with the dynamic channel model, shown in Table 1.

3.2 Performance metrics and problem statements

In general, when applying data broadcasting to provide information services with a time constraint, the LR is considered. It measures the capability of the system to satisfy the

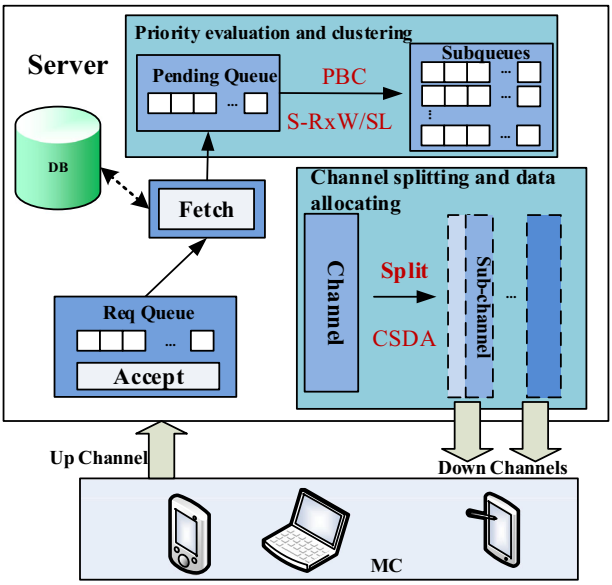

Figure 2 The system architecture

Table 1 Definitions of notations

Notation	Definition
N	The total number of data items on the server
d_i	Let d_i denote the *i-th* data item, $i \in \{1, 2 \ldots N\}$
$Size_{d_i}$	Let $Size_{d_i}$ denote the size of d_i
D	Let D denote the set of data items, $D = \{d_1, \ldots d_i, \ldots d_N\}$
Req_i	Let Req_i denote the *i-th* request, the Req_i characterized by a three-tuple, $Req_i = \{\partial_i, t_i, ID_{i,j}\}$ where ∂_i is an absolute deadline, t_i is the arrival time, and $ID_{i,j}$ is the *ID* of d_j request by Req_i
RQ	Let RQ denote the request queue, $RQ = \{Req_1, Req_2 \ldots Req_i, \ldots\}$
PQ	Let PQ denote the pending queue, $PQ = \{d_1, d_2, \ldots d_i, \ldots\}$
B_k	Let B_k denote the set of data items broadcast in cycle k, where $B_k = \{d_1, \ldots d_i, \ldots, d_l\}$ and l is determined by the S-RxW/SL algorithm
SC	Let SC denote the set of sub-channels, $SC = \{c_1, \ldots c_i, \ldots c_n\}$ where c_i denote the *i-th* sub-channel, and the value of n is dynamic and determined by clustering results
Bw	Let Bw denote the bandwidth of the origin channel C, $Bw = \sum_{i=1}^{n} bw_{c_i}$ where bw_{c_i} is the bandwidth of c_i and bw_{c_i} is dynamic
BQ	Let BQ denote the set of sub broadcast queue, $BQ = \{bq_1, \ldots bq_i, \ldots bq_n\}$ where bq_i is the *i-th* sub broadcast queue, $bq_i = \{d_{i,1}, d_{i,1}, \ldots d_{i,j}, \ldots\}$ where $d_{i,j}$ is the *j-th* data item in bq_i
R_{d_i}	Let R_{d_i} denote the number of requests of d_i
δ_{d_i}	Let δ_{d_i} denote the set of request of d_i, $\delta_{d_i} = \{Req_1, Req_2 \ldots Req_i, \ldots\}$
W_{d_i}	Let W_{d_i} denote the longest waiting time of d_i
SL_{d_i}	Let SL_{d_i} denote the number of lost requests if server broadcast d_i at system time t.
$Weight_{d_i}$	Let $Weight_{d_i}$ denote the priority of d_i, which is determined by R_{d_i}, W_{d_i}, and SL_{d_i}

deadlines of the requests. The formal definition of LR is given in **Definition 1**. An algorithm with a lower LR exhibits a higher performance, and one of our objectives in this work is to generate a broadcast schedule with a lower LR.

Definition 1 (loss rate) The loss rate, denoted by LR, is the ratio of the number of requests missing the deadlines to the total number of requests. Let $Led_{d_i.k}$ denote the number of lost requests and $S_{d_i,k}$ denote the number of successful requests of d_i in cycle k. Thus, the LR in M broadcast cycles is

$$LR = \frac{L_{k<M}}{L_{k<M} + SC_{k<M}} = \frac{\sum_{k=1}^{M} \sum_{i=1}^{N} Led_{d_i.k}}{\sum_{k=1}^{M} \sum_{i=1}^{N} Led_{d_i.k} + \sum_{k=1}^{M} \sum_{i=1}^{N} S_{d_i,k}}. \quad (1)$$

This research is devoted to addressing some issues related to the dynamic channel model in an on-demand data broadcast, and the key challenge is the dynamic adjustment of the channel to adapt to real-time client requests. Accordingly, we investigate the ODCA problem, which aims at determining a reasonable and effective approach for dynamic channel adjustment. Formally, we define the problem ODCA as follows: given a set $RQ = \{Req_1, Req_2 \ldots Req_i, \ldots\}$ of requests with associated data items set $D = \{d_1, \ldots d_i, \ldots d_N\}$, and an original channel, the ODCA problem is to determine an approach for mining the characteristics of D and splitting the original channel into a set of sub-channels $SC = \{c_1, \ldots c_i, \ldots c_n\}$.

4 Channel dynamic adjustment method

In this section, we present the CDAM for real-time and on-demand broadcast. The framework of the CDAM is depicted in Figure 3, which is detailed as follows. There are three main algorithms—the S-RxW/SL algorithm for priority evaluation and selection of data items, WSAC for mining and clustering of data item characteristics, and CSDA algorithm for channel splitting and data allocation. We first present the overall schedule process of the on-demand data broadcast based on CDAM, and subsequently present the S-RxW/SL, WSAC, and CSDA.

Based on the CDAM, the overall scheduling scheme is detailed in **Schedule 1**.

Schedule 1: Scheduling scheme

for each time slot t within the running time T **do:**
if a new Req_i request d_i is submitted at time (t-1, t] **then**
  if d_i is in the BQ **or** PQ, **then** add Req_i to δ_{d_i} **end if**
end if
if a data item d_i is broadcast completely at time t **then**
  delete d_i from BQ, and delete the requests of d_i from RQ;
end if
if $BQ = \varnothing$ **then**
  fetch the data items according to RQ to generate the PQ;
  Apply algorithm S-RxW/SL to evaluate the priority of d_i and select data items to generate B_k;
  Apply algorithm WSAC for mining and clustering B_k into BQ;
  Apply algorithm CSDA for splitting the original channel with BQ to generate SC and allocate bd_i into c_i;
end if
Broadcast;
time slot $t = t + 1$;
end for

4.1 S-RxW/SL algorithm

As mentioned above, priority is one of the most important characteristics of a data item. In order to evaluate the priority of a data item accurately and offer an effective parameter to the WSAC algorithm, we developed the S-RxW/SL algorithm for priority evaluation and selection of a data item. In this section, we first introduce the S-RxW/SL, and subsequently present the implementation of S-RxW/SL in detail.

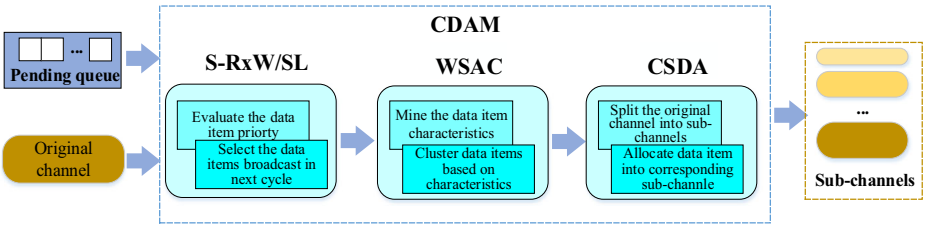

Figure 3 The CDAM framework

Most previous studies on on-demand data broadcast scheduling have focused on the priority evaluation of data items [9, 10, 25, 30]. The priority of a data item implies the extent of emergence of a data item. The main process of a data broadcast scheduling algorithm consists of assigning a priority to each data item and constructing a broadcast queue using the priority. The higher priority data items are preferred for broadcast. The factors affecting the priority evaluation of a data item generally include the data item size [30], request longest waiting time [25], number of requests [9], and request deadline [10]. The analysis of the impact of these four factors on the priority of a data item is illustrated in Table 2.

Based on the above analysis, we developed the S-RxW/SL algorithm, which considers the aforementioned four factors to evaluate the priority of a data item. In the S-RxW/SL, the request number R_{d_i} and the longest waiting time W_{d_i} of d_i are used to evaluate $Weight_{d_i}$ directly, the data size $size_{d_i}$ and the request deadline ∂_i are reflected by system loss SL_{d_i}. Specifically, $Weight_{d_i}$ is obtained using the following equation.

$$Weight_{d_i} = \frac{R_{d_i}{}^i \times W_{d_i}}{SL_{d_i}}, \tag{2}$$

Where SL_{d_i} is the number of lost requests if the server broadcasts d_i at system time t. Further, SL_{d_i} is obtained as follows:

$$SL_{d_i} = \sum_{\partial_j < t + \frac{size_{d_i}}{Bw}, j \neq i} 1 \tag{3}$$

Further, in Eq. (2), it is possible that $SL_{d_i} = 0$; subsequently, we can use $R_{d_i}{}^i \times W_{d_i}$ to evaluate the priority of d_i. Finally, $Weight_{d_i}$ is obtained as follows:

$$Weight_{d_i} = \begin{cases} \dfrac{R_{d_i}{}^i \times W_{d_i}}{SL_{d_i}} & if \;\; SL_{d_i} \neq 0 \\ R_{d_i}{}^i \times W_{d_i} & if \;\; SL_{d_i} = 0 \end{cases} \tag{4}$$

Equation (4) provides the method for calculating $Weight_{d_i}$, however, it is possible that the PQ contains a large number of data items that cannot be broadcast in a cycle. In that case, to

Table 2 Scheduling factor analysis

Parameter	Analysis
Data item size	The larger the data item, the longer the broadcast time. In other words, large data items consume more channel resources. Other things being equal, in order to reduce the AAT and LR, the size of the data item is inversely proportional to the request priority.
Longest waiting time	In order to avoid the starvation of a cold data item, reduce the AAT, and reduce the possibility of request loss, the waiting time of requests must be considered. The longer the waiting time, the higher the priority of the data item.
Request number	Request number is the key indicator to evaluate the heat of a data item, to reduce the overall AAT and LR, the greater request number of the data item means the higher priority.
Deadline	In order to achieve a lower LR, the deadline of the request must be considered. The deadline depicts the urgency of the request; the shorter the deadline, the more urgent the request, and the higher the priority of the data item requested by the request.

ensure the effectiveness of WSAC and CSDA, we should select a set of higher priority data items. Otherwise, those data items that are not broadcast will become the cluster noise. Finally, the S-RxW/SL algorithm is given in detail in **Algorithm 1**.

Algorithm 1: The S-RxW/SL algorithm

Input: PQ;
Output: B;
1. Initialize $B = \varnothing$, $index = 0$, $sumsize = 0$
2. **for** each $d_i \in PQ$:
3. Calculate $Weight_{d_i}$ using Equation (4);
4. $index = index + 1$
5. **end for**
6. Schedule the PQ by descending order of $Weight_{d_i}$;
7. **for** $i = 1$ to $index$:
8. **if** $sumsize < Bw \times T$ **then**:
9. $sumsize = sumsize + size_{d_i}$;
10. add d_i into B, delete d_i from PQ;
11. **else break;**
12. **end if**
13. **end for**
14. **output** B.

4.2 WSAC algorithm

There are two critical characteristics of a data item for adaptive channel adjustment in the dynamic channel model: data item priority and data item size. On the one hand, the priority of a data item determines the extent of emergence of a data item. The data items with higher priority are preferred for broadcast to ensure high system performance. On the other hand, the data item size is related to broadcast time. In order to reduce the AAT and improve the channel utilization, the bigger the size of the data item, the wider the bandwidth required. In conclusion, to achieve a high performance, the channel adapts to the corresponding data item characteristics required. In the on-demand data broadcast, the client demands and data item size have a large degree of diversity, which result in the dynamic diversity of data item characteristics. Based on the above analysis, we developed the WSAC algorithm for mining the characteristics of data items and clustering them. In this section, we first analyze the relationship between the data item characteristics and channel bandwidth, and subsequently present the WSAC in detail.

4.2.1 Data item characteristics and broadcast channel

This section analyzes the relationship between the data item characteristics and broadcast channel. Intuitively, the two most important characteristics of a data item are: data item priority and data item size. The key feature of WSAC is to determine the optimal channel adjustment scheme based on the data item priority and size, which facilitates the adaptation of the adjusted channels to the current data item characteristics. We first introduce the definition of broadcast throughput to quantify the fitness of the broadcast channel and data item characteristics, and subsequently use two examples to further illustrate and analyze it.

Definition 2 (broadcast throughput) The broadcast throughput, denoted byTP, is the sum of the data item priority broadcast by all the channels. Let $Weight_{d_i,c_i,k}$ denote the priority of d_i, which is broadcast in sub-channelc_i, in the *k-th* broadcast cycle. Thus, the broadcast throughput $tp_{c_i,k}$of c_i in the *k-th* cycle is calculated as follows:

$$\begin{aligned} tp_{c_i,k} &= Weight_{d_1,c_i,k} + Weight_{d_2,c_i,k} + \ldots \\ &= \sum_{d_i \in c_i} Weight_{d_j,c_i,k} \end{aligned} \tag{5}$$

Subsequently, the broadcast throughput TP_k of all the channels in the *k-th* cycle is calculated as follows:

$$\begin{aligned} TP_k &= tp_{c_1,k} + tp_{c_2,k} + \ldots + tp_{c_n,k} \\ &= \sum_{i=1}^{n} tp_{c_i,k} \end{aligned} \tag{6}$$

Example 1. Given a set $D = \{d_1, d_2, \ldots d_6\}$ data items and a set of sub-channels $SC = \{c_1, c_2\}$, where $Weight_{d_1} = \ldots = Weight_{d_6} = 1$, $size_{d_1} = size_{d_2} = size_{d_2} = 5$, $size_{d_4} = size_{d_5} = size_{d_6} = 1$ and $bw_{c_1} = 5, bw_{c_2} = 1$, we assume that the broadcast cycle length is 3 s. As illustrated in Figure 4a, if we cluster those data items with similar $size_{d_i}$, d_{1-3} is clustered into bq_1 and d_{4-6} is clustered into bq_2. Finally, allocating bq_1 and bq_2 to $c_1$and $c_2$respectively. There is no bandwidth waste in this scheduling scheme and $TP_k = 6$. However, random allocation of data items will inevitably lead to a waste of bandwidth. Figure 4b shows the worst case scenario, where d_{1-5} is allocated to c_1 and d_6 is allocated to c_2. In this case, 2 units bandwidth will be wasted in c_1. Owing to d_6 requiring 5 units bandwidth to broadcast, d_6 cannot be broadcast in $c_2$for a broadcast cycle. This will lead to the failure of c_2. Based on the above comparison, it is wise to cluster the data items with similar size, and adjust the broadcast channel according to the data item size.

Example 2. Given a set $D = \{d_1, d_2, \ldots d_8\}$ of data items and a set of sub-channels $SC = \{c_1, c_2\}$, where $size_{d_1} = \ldots = size_{d_8} = 1, bw_{c_1} = bw_{c_2} = 1$, $Weight_{d_{1-4}} = 5, Weight_{d_{5-8}} = 1$, we assume that the broadcast cycle length is 3 s. As illustrated in Figure 4c, if we cluster

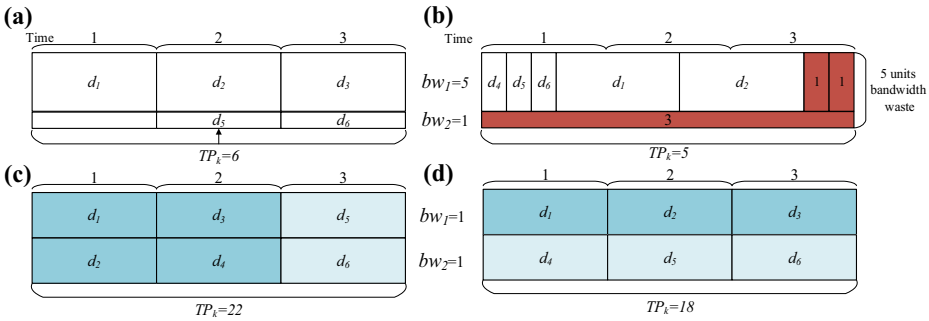

Figure 4 Example of data item characteristics and braodcast channel analysis

d_{1-8} with priority equalization, subsequently, $d_{1,3,5,7}$ is clustered into bq_1 and $d_{2,4,6,8}$ is clustered into bq_2. Finally, allocating bq_1 and bq_2 into c_1 and c_2 respectively, d_7, d_8 will not be broadcast owing to the cycle length of 3 s, and $TP_k = 22$. Figure 4d illustrates the worese case where the priority equalization is not considered, i.e., allocating d_{1-4} to $c_1$and allocating $d_{5-8}$to c_2, d_4, d_8 will not broadcast. In this case, $TP_k = 18$. It is evident that it will benefit the broadcast efficiency to cluster the data items using priority equalization.

4.2.2 WSAC algorithm

Based on the above analysis, we describe the proposed WSAC alogrithm in detail. As illustrated in Figure 5, it comprises three parts: clustering data items by $Weight_{d_i}$; clustering data items by $Size_{d_i}$; $Weight_{d_i}$ equalization. Each part is detailed as follows.

Part 1 (Clustering data items by $Weight_{d_i}$): In order to balance the priority of data items, the improved knowledge discovery by accuracy maximization (KODAMA) [29] algorithm is used for automatically clustering $B_k = \{d_1, \ldots d_i, \ldots, d_l\}$, which is generated by the S-RxW/SL into a set of subsets $S = \{s_1, \ldots s_i, \ldots s_m\}$, where $s_i = \{d_1, \ldots d_i, \ldots\}$ and m are determined by KODAMA. After the clustering, the average priority $AWeight_{s_i}$ is calculated using Eq. (7), where $|s_i|$ denotes the number of data items existing in s_i. Subsequently, S is scheduled according to the descending order of $AWeight_{s_i}$, i.e., s_1 has the highest average priority. The KODAMA is detailed in **Algorithm 2**.

$$AWeight_{s_i} = \frac{\sum\limits_{d_i \in s_i} Weight_{d_i}}{|s_i|} \tag{7}$$

Part 2 (Clustering data items by $Size_{d_i}$): The set of subsets S is generated after **Part 1**. Each data item contained in s_i has a similar priority. The subsequent step is using the KODAMA to cluster each s_i into a set of subsets $s_i = \left\{s_{i,1}, \ldots s_{i,j}, \ldots s_{i,x^i}\right\}$, where $s_{i,j}$ denotes the *j-th* subset of s_i. Subsequently, the average data size $ASize_{s_{i,j}}$ is calculated using Eq. (8), where $|s_{i,j}|$ denotes the number of data items existing in $s_{i,j}$. Finally, $s_i$is scheduled according to the descending order of $ASize_{s_{i,j}}$, i.e., $s_{i,1}$ has the largest average data size in s_i.

$$ASize_{s_{i,j}} = \frac{\sum\limits_{d_i \in s_{i,j}} size_{d_i}}{|s_{i,j}|} \tag{8}$$

Part 3 ($Weight_{d_i}$equalization): In the above two parts, the data items are clustered by priority and size, and a set of subsets $S = \{s_1, \ldots s_i, \ldots s_m\}$is genrated, where

$s_i = \{s_{i,1}, \ldots s_{i,j}, \ldots s_{i,x^i}\}$. Each $s_i$has the following properties: $AWeight_{s_i}$ between s_i is different, subsequently, the average priority between $s_{a,i}$ and $s_{b,i}$ is different, however, the data items in $s_{a,i}$ are similar in priority. $Weight_{d_i}$ equalization is required to balance the priority of data items. As shown in Figure 5, $BQ = \{bq_1, \ldots bq_i, \ldots bq_n\}$ is genarated by the combined operation. The *i-th* subset from each s is selected to form bd_i.

`

Finally, the WSAC algorithm is given in detail in **Algorithm 2**.

Algorithm 2: The WSAC algorithm

Input: B_k

Output: BQ;

1. Initialize the $BQ = \varnothing$;
2. Clustering the B_k into a set of subsets S by $Weight_{d_i}$ using ***KODAMA***;
3. ***KODAMA*** **:**
4. Assign each d_i in B_k to a class defined in the class indicator vector $T = \{t_1, \ldots t_i, \ldots t_l\}$;
5. Determine the K value using *Optimal K Strategy*;
6. A 10-fold cross-validation procedure is performed on the basis of the classes defined in T using k-means algorithm;
7. A record of the predicted class labels for each sample is subsequently stored in $Z_T = \{z_1, \ldots z_i, \ldots z_l\}$;
8. The global accuracy V_T is calculated by summing the number of correctly classified samples and dividing this number by the total number of samples;
9. A new class indicator vector $V = \{v_1, \ldots v_i, \ldots v_l\}$ is created by randomly swapping the class labels of the misclassified samples with the predicted class labels stored in Z_T;
10. A 10-fold cross-validation procedure is performed on the basis of the classes defined in V;
11. Calculate the relative accuracy value A_V
12. **if** $A_V > A_T$ **then**: $A_T = A_V$ **;** $T = V$ **;** $Z_T = Z_V$ **; end if**
13. **if** $A_T \neq A_V$ **then goto 8;**
14. **end** ***KODAMA*****;**
15. **for** each $s_i \in S$:
16. Calculate the $AWeight_{s_i}$ by Equation (7);
17. **end for**
15. Schedule the S by descending order of $AWeight_{s_i}$;
18. **for** each $s_i \in S$:
19. Clustering the s_i into a set of subsets $s_i = \{s_{i,1}, \ldots s_{i,j}, \ldots s_{i,x^i}\}$ by $size_{d_i}$ using ***KODAMA***;
20. **for** each $s_{i,j} \in s_i$:
21. Calculate the $ASize_{s_{i,j}}$ by Equation (8);
16. Schedule the s_i by descending order of $ASize_{s_{i,j}}$;
22. **end for**
23. **end for**
24. **for** $j = 1$ to n **:**
25. Initialize the bq_j ;
26. **for** each $s_i \in S$:
27. Add the $s_{i,j}$ into bq_j
28. **end for**
29. **end for**

4.2.3 Optimal K strategy

The k-means algorithm is used as a supervised classifier in the WSAC. A limitation of the k-means algorithm is the determination of the number of clusters (the value of K) without prior knowledge [18]. In an on-demand data broadcast system, the characteristics of a data item are

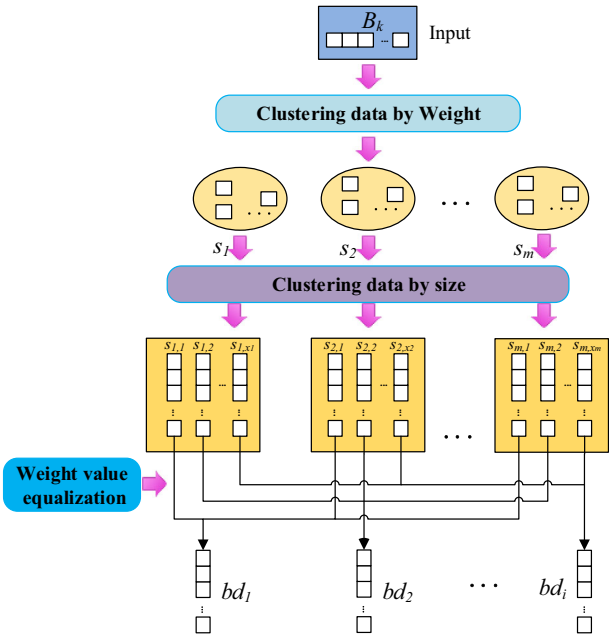

Figure 5 General process of WSAC

determined by real-time requests; thus, it is necessary to remove the limitation of specifying the value of K to obtain a consistent and reliable clustering result. Moreover, various methods have been devised to solve this problem, including Forgy's method, simple cluster seeking method, robust k-means algorithm etc. Especially, the robust k-means algorithm is useful in practice because it not only addresses the sensitivity in selecting the initial cluster centers, but also is resilient to the initial value of K. In this section, we propose the optimal K strategy based on the robust k-means algorithm, which can automatically determine the optimal value of K to ensure the accuracy of WSAC.

It is well known that the inter-cluster dissimilarity d_{inter}should be as large as possible and the intra-cluster dissimilarity d_{intra}should be as small as possible. In other words, d_{inter} should be greater thand_{intra}. It is reasonable that the two clusters should be merged when the dissimilarity between the two clusters is less than the minimum inter-cluster dissimilarity. According to [18], the minimum inter-cluster dissimilarity should be set to $\bar{d}/_2$, where $\bar{d}$ denotes the average inter-cluster dissimilarity. Further, $\bar{d}$ and d_{inter} are calculated using Eqs. (9) and (10), respectively.

$$\bar{d} = \frac{1}{A_k^2} \sum_{i=1}^{k} \sum_{j=1}^{k} \left|\left|m_i - m_j\right|\right|^2 \tag{9}$$

$$d_{inter}^{i,j} = \left|\left|m_i - m_j\right|\right|^2 \tag{10}$$

where $m_i$and m_j represent the cluster centers and A_k^2 indicates the number of pairwise cluster centers.

The procedure of optimal K strategy is formalized in **Strategy 1**.

Strategy 1: Optimal K strategy

1. Given a set of N samples, assign each sample to a different cluster ϕ_i, subsequently the samples are assigned to a set of clusters $\Omega = \{\phi_1, ...\phi_i, ...\phi_K\}$, set $K = N$;
2. Calculate the average inter-cluster dissimilarity $\overline{d}$ using Equation (9);
3. **for** each $\phi_i \in \Omega$:
4. **for** each $\phi_j \in \Omega$ and $i \neq j$:
5. Calculate the inter-cluster dissimilarity $d_{inter}^{i,j}$ using Equation (10);
6. **if** $d_{inter}^{i,j} < \overline{d}/2$ **then:** merge ϕ_i and ϕ_j; $K = K - 1$; **end if**
7. **end for**
8. **end for**
9. **if** no cluster merged **then:** return K;
10. **else goto 2; end if**

4.3 CSDA algorithm

The S-RxW/SL evaluates the priority of data items and selects the high priority data items to formB_k; subsequently, the WSAC mines the characteristics of data items inB_k, and clusters $B_k$into a set of sub-broadcast queueBQ. Each bq_i in BQ has the following properties: each $d_i$in bq_i has a similar$size_{d_i}$, and high priority data items are evenly distributed in eachbq_i. Finally, the CSDA is proposed to dynamically adjust the broadcast channel to adapt to BQ. Splitting the original channel in real-time according to the characteristics of the data items allows the broadcast to adapt to the varying mobile networks and improve the efficiency of the real-time on-demand data broadcast system.

Based on the above analysis, we describe the proposed CSDA algorithm in detail. It comprises two parts: channel splitting and data allocation.

Part 1 (Channel splitting): Channel splitting contains two sub-problems: the determination of the number of sub-channels and the determination of the bandwidth of each sub-channelc_i. In CSDA, the number of sub-channels is determined by $BQ = \{bq_1, \ldots bq_i, \ldots bq_n\}$. In other words, the number of sub-channels is n, where n is the number of subsets contained in BQ. In addition, in order to improve the channel utilization, the bandwidth of $c_i$should adapt to the size of the correspondingbd_i. Subsequently, $bw_{c_i}$is calculated as follows:

$$bw_{c_i} = \frac{Size_{bq_i}}{SunSize} = \frac{\sum_{d_i \in bq_i} size_{d_i}}{\sum_{d_i \in BQ} size_{d_i}} \tag{11}$$

Part 2 (Data allocation): The number and bandwidths of sub-channels were determined in **Part 1**. The final operation is assigning $bd_i$to the corresponding sub-channelc_i, and subsequently scheduling each bd_i according to the descending order of $Weight_{d_i}$ to improve the broadcast efficiency.

Finally, the CSDA algorithm is detailed in **Algorithm 3**.

Algorithm 3: The CSDA algorithm

Input: *BQ, C;*
Output: *SC*;
for each $bq_i \in BQ$:
  Initial a sub-channel c_i ;
  Calculate bw_{c_i} by Equation (11);
end for
for each $bq_i \in BQ$:
  Schedule the bq_i by descending order of $Weight_{d_i}$;
  Assign bq_i to c_i for data broadcast.
end for

5 Experiments and analysis

In this section, we compare the CDAM with the algorithms described in Section 2: RxW, TOSA, MPCF, and SLLH. In order to verify the advantages of the dynamic channel model compared to the fixed channel model, the RxW for fixed single-channel architecture, and TOSA, MPCF, SLLH for fixed multi-channel architecture are selected as the contrast algorithms. Further, RxW, MPCF, SLLH are pull-based algorithms, and TOSA is a push-based algorithm. The experimental environment is summarized in Section 5.1. The impacts of the varying deadline, request distribution, and data size are discussed in Sections 5.2, 5.3, and 5.4, respectively.

5.1 Experimental environment

In order to simulate a high-load on-demand data broadcast environment, we utilize real traces collected from the FIFA World Cup 1998 Website [33] to simulate the requests made by clients as follows. The real-time requests are those for information, such as the current scores for live games. We use the trace of day 38, which contains more than 7 million requests for 4923 distinct Web pages, with an access pattern following the Zipf distribution. We assume the average deadline to be 60 s, and assign using uniform, exponential, and fixed distributions. The request arrival rate is 100 requests/s and it is assigned using the Zipf, uniform, and normal distributions. We also assume that the size of data items is dynamic and the average size is 1 unit, assigned using the uniform distribution. The default values for the parameters used in our simulations are shown in Table 3.

5.2 Impact of varying data request

We begin our comparison of the performances of the CDAM, RxW, TOSA, MPCF, and SLLH broadcast scheduling algorithms for an on-demand broadcast environment by measuring the LR as the request varies in terms of arrival rate. The request arrival rate determines the load of the system; hence, we first discuss the impact of varying the request arrival rate on the

Table 3 Experiment parameter settings

Parameter name	Distribution	Default	Variable
Broadcasting cycle	-	20 s	*Y*
Total bandwidth	-	100 unit/s	*Y*
Sub-channel number		5	*Y*
Run time	-	3600 s	*N*
Deadline	Uniform	60 s	*Y*
Data item size	Uniform	1 unit	*Y*
Request arrival rate	Zipf	100	*Y*
Total request number	-	300,000	*N*
Number of data items	-	4923	*N*

performance of the algorithms. The value of the request arrival rate is set from 20 to 180 requests per time slot to simulate low and high system loads. Other parameters are set to the default values in Table 3. The results are illustrated in Figure 6.

As illustrated in Figure 6, the LR of all the algorithms increases as the request arrival rate increases. This is expected since the system can only serve one data item at any one time slot. A higher arrival rate indicates that more real-time requests will remain in the request queue, which in turn will result in a higher LR. The better performance between the RxW (pull-based algorithm) and TOSA (push-based algorithm) depends on the arrival rate. At the lowest arrival rate, the RxW has a lower LR than TOSA, whereas at a high arrival rate, the RxW has a higher LR than TOSA. Overall, the multi-channel and pull-based algorithms (MPCF, SLLH, and CDAM) outperform the single channel (RxW) or push-based algorithms (TOSA). This demonstrates that neither the single-channel based algorithm nor the push-based algorithm can perform efficiently for on-demand broadcast, especially at a high request arrival rate. SLLH is similar to MPCF when the arrival rate is low and it outperforms RxW and TOSA. When the arrival rate more than 100 requests per time slot, SLLH outperforms the MPCF and performs similarly to CDAM. Among all the algorithms, the CDAM has the lowest LR whether the arrival rate is low or high. This phenomenon verifies the effectiveness of the dynamic channel model based algorithms CDAM.

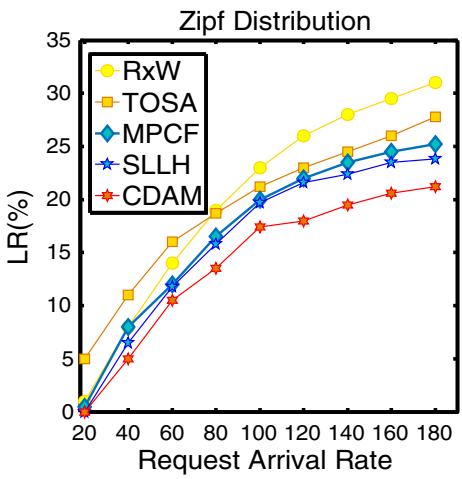

Figure 6 The impact of varying request arrival rate

5.3 Impact of varying deadline

In this section, we discuss the impact of varying the deadline on the performance of the algorithms. Similar to the study in [4], deadlines are assigned using uniform, exponential, and fixed distributions. The value of deadline is set from 10 s to 210 s to simulate high and low urgent degrees. Other parameters are set to the default values in Table 3. The results are illustrated in Figure 7.

On the basis of the results shown in Figure 7, we can propose the following observations. The LR of all the algorithms decreases as the deadline increases. Specifically, the LR of all the algorithms decreases significantly as deadline increases from 10 s to 110 s, and decreases slightly as deadline increases from 135 s to 210 s. This phenomenon indicates that the deadline is the determining factor of LR when the deadline is urgent. The LR is the greatest for the exponential deadlines and the smallest for fixed deadlines. In general, pull-based algorithms (RxW, MPCF, SLLH, and CDAM) outperform the push-based algorithm (TOAS) because the data items broadcast by the push-based algorithm are determined by prior knowledge and cannot satisfy the real-time requests of clients, and multi-channel based algorithms (TOSA, MPCF, SLLH, and CDAM) outperform the single-channel based algorithm (RxW) because a multi-channel based algorithm has the advantage of parallel broadcasting. Our CDAM always achieves the lowest LR, irrespective of the type of deadline distribution; this result verifies the effectiveness of the CDAM. Compare to the other algorithms, CDAM dynamically adjusts the broadcast channel to adapt to the real-time broadcast environment; therefore, it can adapt to different broadcast demands and always achieves the best LR with different deadline urgencies and distributions. Especially in the case of an urgent deadline, CDAM significantly outperforms other contrast algorithms. This reveals that the CDAM is a better option for minimizing the LR under an urgent deadline.

5.4 Impact of varying data item size

The size is an important characteristic of a data item, which be considered as a cluster index in the CDAM. In this section, experiments are carried out to verify the impact of varying the data item size on the performance of the algorithms. We assume that the size of data item is assigned a uniform distribution and set from 0.25 to 2.5 units. Other parameters are set to the default values in Table 3. The results are illustrated in Figure 8.

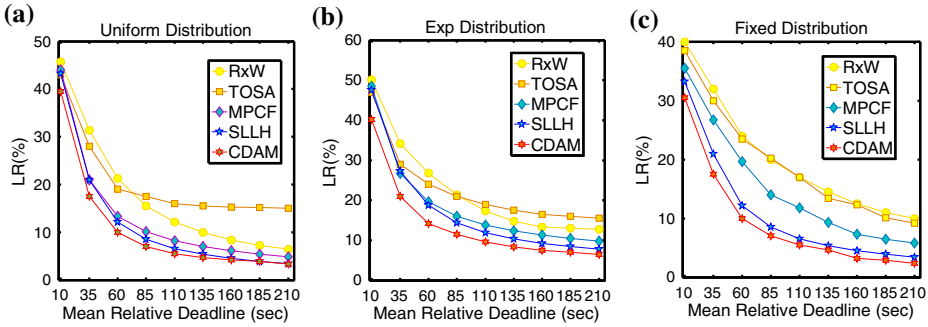

Figure 7 Loss rates under different request deadline distributions

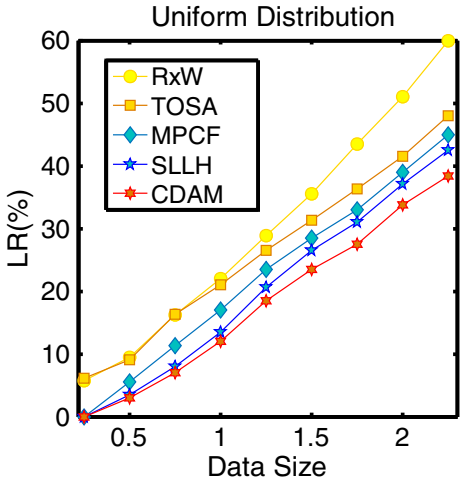

Figure 8 The impact of varying data item size

As shown in Figure 8, the LR of all the algorithms increases slightly as the data item size increases from 0.25 to 1, and increases significantly as the data item size increases from 1 to 2.5. This is consistent with the intuition that larger data item size indicates more time taken to broadcast a data item; subsequently, more real-time requests will miss their deadlines, which in turn results in a higher LR. The advantages of parallel broadcast of multi-channel based algorithms are fully reflected in the case where the data item size is large, i.e., the RxW (single-channel based algorithm) has the highest LR when data item size is more than 0.75. The CDAM considers the data item size as an important channel adjustment characteristic, and adjusts the sub-channel bandwidth according to $Size_{bq}$; therefore, it always determines the most suitable sub-channel bandwidth for data broadcast, which in turn results in the lowest LR compared to the other fixed channel model based algorithms. With the increase of data item size, the advantage of CDAM is more evident. These results indicate that the CDAM is the better option in a dynamic data item size broadcast environment.

5.5 Impact of varying Zipf access skew coefficient

We assume that the access pattern follows the Zipf distribution, which is a typical model for Web data with non-uniform access patterns. Let θ denote the Zipf access skew coefficient. The probability that data item d_i is requested is $P_i = \frac{\left(1/i\right)^{\theta}}{\sum_{i=1}^{N}\left(1/i\right)^{\theta}}$, where $\theta \in [0, 1]$. Different values of the skew coefficient θ yield different Zipf distributions. For $\theta = 0$, the Zipf distribution reduces to a uniform distribution. However, the distribution becomes increasingly skewed as θ increases. In all the previous comparisons, the default value of θ is 0.5, and subsequently we determine the effect of θ on LR. The value of θ is varied from 0 to 1, and the other parameters are set to the default values in Table 3.

As illustrated in Figure 9, the possibility of accessing smaller data increases as θ changes from 0 to 1. Small data objects consume less bandwidth and the system loading decreases accordingly. Therefore the LR of all the algorithms decreases gradually as θ increases. It is evident that the LR decreases at a much slower rate for the push-based algorithm TOSA than

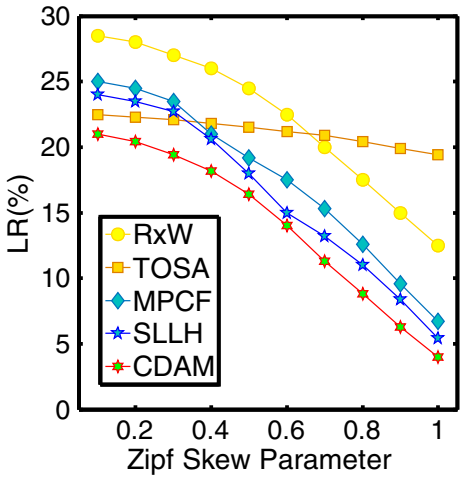

Figure 9 Loss rates under different request deadline distributions

the pull-based algorithms. The TOSA has a lower LR than RxW, MPCF, and SLLH when θ is small; whereas at a greater θ, the TOSA has a higher LR than RxW, MPCF, and SLLH. This can be attributed to the fact that, by considering the priority, it is more likely for a selected data object to satisfy multiple requests before their deadlines as the access pattern becomes highly skewed for pull-based algorithms. On the whole, we can observe that CDAM outperforms the other contrast algorithms and has the fastest decrease rate of LR. This confirms that, in the case of obvious characteristics of data (highly access pattern skew), CDAM clusters more accurately, and it can accurately adjust the broadcast channel.

6 Conclusions and future work

In this paper, we have discussed the fixed channel model problem caused by the dynamic diversity of data characteristics and client demand. The problem is that the fixed channel model, including single channel architecture and fixed multi-channel architecture, cannot adjust the number and bandwidths of broadcast channels dynamically to adapt to the real-time broadcast environment. Accordingly, we have investigated the problem of ODCA and proposed a channel dynamic adjustment method. There are three algorithms in CDAM: the S-RxW/SL algorithm for priority evaluation and selection of data items, the WSAC for mining and clustering of data item characteristics, and the CSDA algorithm for channel splitting and data allocation. Extensive simulation experiments have been conducted to evaluate the performance of CDAM, allowing a number of conclusions to be drawn as follows.

(1) Compared to the fixed channel model, the dynamic channel model proposed in this paper has a stronger adaptability and outperforms the previously proposed algorithms. This demonstrates that adjusting the broadcast channel dynamically benefits the broadcast efficiency.
(2) A set of factors including request arrival rate, deadline, data item size, and access pattern skew affect the data broadcast. A scheduling algorithm that considers a more comprehensive set of factors enhances the performance.

In this paper, we assume that a client can only request one data item at a time. In a future study, we will investigate the scheduling problem when a client can simultaneously request more than one data item.

Acknowledgements This work is partially supported by National Natural Science Foundation of China (61572369, 61471274).

References

1. Aksoy, D., Franklin, M.: RxW: a scheduling approach for large scale on-demand data broadcast. IEEE/ACM Trans. Networking. **7**(6), 846–860 (1999)
2. Anticaglia, S., Barsi, F., Bertossi, A.A., et al.: Efficient heuristics for data broadcasting on multiple channels. Wirel. Netw. **14**(2), 219–231 (2008)
3. Chen, J., Lee, V.C.S., Liu, K., et al.: Efficient processing of requests with network coding in on-demand data broadcast environments. Inf. Sci. **232**(5), 27–43 (2013)
4. Chung, Y., Chen, C., Lee, C.: Design and performance evaluation of broadcast algorithms for time-constrained data retrieval. IEEE Trans. Knowl. Data Eng. **18**(11), 1526–1543 (2008)
5. Dewri, R., Ray, I., Ray, I., et al.: Optimizing on-demand data broadcast scheduling in pervasive environments. In: EDBT 2008, International Conference on Extending Database Technology, Nantes, 25–29 Mar 2008, pp. 559–569. Proceedings, DBLP (2008)
6. Dykeman, H.D., Wong, J.W.: A performance study of broadcast information delivery systems. In: INFOCOM '88. Networks: Evolution Or Revolution, Proceedings. Seventh Joint Conference of the IEEE Computer and Communcations Societies, IEEE, New Orleans, 27–31 Mar 1988, pp. 739–745. IEEE (2002)
7. Fang, Q., Vrbsky, S.V., Dang, Y., et al.: A pull-based broadcast algorithm that considers timing constraints. In: International Conference on Parallel Processing Workshops. IEEE Computer Society, Montreal, 18 Aug 2004, pp. 46–53. IEEE (2004)
8. Gao, X., Yang, Y., Chen, G., et al.: Global optimization for multi-channel wireless data broadcast with AH-tree indexing scheme. IEEE Trans. Comput. **65**(7), 2104–2117 (2016)
9. He, P., Shen, H.: On-demand multimedia data broadcast in MIMO wireless networks. Physical Communication. **20**, 1–16 (2016)
10. He, P., Shen, H., Tian, H.: On-demand data broadcast with deadlines for avoiding conflicts in wireless networks. J. Syst. Softw. **103**(C), 118–127 (2015)
11. Hu, W., Xia, C., Du, B., Wu, M.: An on-demanded data broadcasting scheduling considering the data item size. Wirel. Netw. **21**(1), 35–56 (2015)
12. Hu, W., Fan, C., Luo, J., et al.: An on-demand data broadcasting scheduling algorithm based on dynamic index strategy. Wireless Communications & Mobile Computing. **15**(5), 947–965 (2015)
13. Ji, H., Lee, V.C.S., Chow, C.Y., et al.: Coding-based cooperative caching in on-demand data broadcast environments. Inf. Sci. **385**(C), 138–156 (2017)
14. Jung, H., Chung, Y., Liu, L.: Processing generalized k-nearest neighbor queries on a wireless broadcast stream. Inf. Sci. **188**(4), 64–79 (2012)
15. Kalyanasundaram, B., Velauthapillai, M. On-demand broadcasting under deadline. In: Proceedings of the 11th Annual European Symposium on Algorithms, pp. 313–324. (2003)
16. Lee, W., Hu, Q., Lee, D.: A study on channel allocation for data dissemination in mobile computing environments. Mobile Networks and Applications. **1**(2), 117–129 (1999)
17. Lei, M., Vrbsky, S.V., Xiao, Y.: Scheduling on-demand data broadcast in mixed-type request environments. Comput. Netw. **54**(5), 811–825 (2010)
18. Lei, J., Jiang, T., Wu, K., et al.: Robust K -means algorithm with automatically splitting and merging clusters and its applications for surveillance data. Multimedia Tools & Applications. **75**(19), 1–17 (2016)
19. Li, L.J., Liu, H.F., Yang, Z.Y., et al.: Broadcasting methods in vehicular ad hoc networks. Journal of Software. **21**(7), 1620–1634 (2010)
20. Lim, J.H., Naito, K., Yun, J.H., et al.: Revisiting overlapped channels: Efficient broadcast in multi-channel wireless networks. In: Computer Communications (INFOCOM), 2015 I.E. Conference on. IEEE. (2015)
21. Liu, C.M., Su, T.C.: Broadcasting on-demand data with time constraints using multiple channels in wireless broadcast environments. Inf. Sci. **242**(2), 76–91 (2013)
22. Lu, Z., Shi, Y., Wu, W., et al.: Data retrieval scheduling for multi-item requests in multi-channel wireless broadcast environments. IEEE Trans. Mob. Comput. **13**(4), 752–765 (2014)

23. Lu, Z., Wu, W., Li, W.W., Pan, M.: Efficient scheduling algorithms for on-demand wireless data broadcast. In: IEEE INFOCOM 2016 - 35th Annual IEEE International Conference on Computer Communications, 10–14 Apr 2016. Institute of Electrical and Electronics Engineers Inc. (2016). doi:10.1109/INFOCOM.2016.7524596
24. Lv, J., Lee, V.C.S., Li, M., et al.: Supporting multi-level quality of services in data broadcast systems[J]. Int. J. Sens. Netw. **18**(3/4), 142–153 (2012)
25. Lv, J., Lee, V.C.S., Li, M., et al.: Supporting multi-level quality of services in data broadcast systems. International Journal of Sensor Networks. **18**(3/4), 142–153 (2015)
26. Nawaz Ali, G.G.M., Lee, V.C.S., Chan, E., et al.: Admission control-based multichannel data broadcasting for real-time multi-item queries. IEEE Trans. Broadcast. **60**(4), 589–605 (2014)
27. Ng, J., Lee, V., Hui, C.: Client-side caching strategies and on-demand broadcast algorithms for real-time information dispatch system. IEEE Trans. Broadcast. **54**(1), 24–35 (2008)
28. Saxena, N., Pinottti, M.C.: On-line balanced k-channel data allocation with hybrid schedule per channel. In: Proceedings of the 6th international conference on Mobile data management, Ayia Napa, 9–13 May 2005, pp. 239–246. ACM, New York (2005)
29. Stefano, C., Claudio, L., Leonardo, T.: Knowledge discovery by accuracy maximization. Proc. Natl. Acad. Sci. U. S. A. **111**(14), 5117–5122 (2014)
30. Sun, W., Qin, Y., Wu, J., et al.: Air indexing for on-demand XML data broadcast. IEEE Transactions on Parallel and Distributed Systems. **25**(6), 1371–1381 (2014)
31. Waluyo, A., Srinlvasan, B., Taniar, D., et al.: Incorporating global index multi data placement scheme for multi channels mobile broadcast environment. Lect. Notes Comput. Sci. **3824**, 755–764 (2005)
32. Wang, H., Xiao, Y., Shu, L.C.: Scheduling periodic continuous queries in real-time data broadcast environments. IEEE Trans. Comput. **61**(9), 1325–1340 (2012)
33. World Cup 98 Web Site Access Logs. http://ita.ee.lbl.gov/html/contribute/WorldCup.html[EB/OL] (1998)
34. Wu, X., Lee, V.C.S.: Wireless real-time on-demand data broadcast scheduling with dual deadlines. Journal of Parallel and Distributed Computing. **65**(6), 714–728 (2005)
35. Xuan, P., et. al.: Broadcast on demand - efficient and timely dissemination of data in mobile environments. In: IEEE Real-Time Technology and Applications Symposium, Montreal, 9–11 Jun 1997, pp. 38–48. IEEE (1997)
36. Zheng, B., Wu, X., Jin, X., Lee, D.: TOSA: a near-optimal scheduling algorithm for multi-channel data broadcast. In: Proceedings of the Sixth ACM International Conference on Mobile Data Management (MDM'05), Ayia Napa, 9–13 May 2005, pp. 29–37